

@devopschallengehub



Linux & DevOps Organized Q&A

1. File Operations & Management

Q: Download a file and save as app.tar.gz on a Linux server without UI

- `wget -O app.tar.gz <url>`
- `curl -L <url> -o app.tar.gz`
- *Note: -L follows redirects, -O (wget) or -o (curl) specifies output filename*

Q: Backup /etc/nginx/nginx.conf to /backup/nginx.conf.bak without changing ownership

- `cp -p /etc/nginx/nginx.conf /backup/nginx.conf.bak`
- *The -p flag preserves permissions, ownership, and timestamps*

Q: Move all .log files from /var/tmp to /logs/archive/

- `mv /var/tmp/*.log /logs/archive/`
- *Moves all matching files in one command*

***Q: Delete all files named .tmp older than 7 days in /var/tmp**

- `find /var/tmp -name "*.tmp" -mtime +7 -exec rm {} \;`
- *Alternative: `find /var/tmp -name "*.tmp" -mtime +7 -delete`*
- *-mtime +7 means modified more than 7 days ago*

Q: Safely delete a non-empty directory named temp_builds

- `rm -rf temp_builds`
- *-r = recursive, -f = force (no prompts)*

Q: Avoid accidental `rm -rf *` in production

Quick prevention checklist (one-liners you can set now)

- Make rm interactive (prompts):
alias rm='rm -I'
Put that in ~/.bashrc or ~/.zshrc.
 - Preserve root: modern rm has --preserve-root by default, keep it.
 - Install trash-cli (Ubuntu/Debian):
sudo apt install trash-cli
Then use trash-put instead of rm to send files to the trash.
-

2. File Viewing & Monitoring

Q: Monitor the last lines of a log file continuously

- tail -f /var/log/app.log
- -f *follows file updates in real-time*

Q: View the last 20 lines of a live production log file

- tail -f -n 20 production.log
- *Combines follow mode with line limit*

Q: Show first 20 lines + last 20 lines of a config file

- head -20 file.conf
 - tail -20 file.conf
 - *Run separately or combine:* (head -20; tail -20) < file.conf
-

3. Text Processing & Editing

Q: Replace all occurrences of "staging" with "production" in config.yml in-place

- sed -i 's/staging/production/g' config.yml
- -i = *in-place edit*, g = *global (all occurrences on each line)*

Q: Replace "dev" with "prod" in a config file

- sed -i 's/dev/prod/g' config.yaml

Q: Extract 1st and 3rd column from a CSV file

- cut -d',' -f1,3 file.csv
- -d = *delimiter*, -f = *fields to extract*

Q: List unique IPs from access log (1st column)

- awk '{print \$1}' access.log | sort | uniq
- Alternative: awk '{print \$1}' access.log | sort -u

- Assumes IP is in first column; sort is required before uniq
-

4. Permissions & Ownership

Q: Log file /var/log/app.log must be writable only by appuser, readable by group appgroup, no access for others

- `chown appuser:appgroup /var/log/app.log`
- `chmod 640 /var/log/app.log`
- `640 = rw-r-----` (owner: read/write, group: read, others: none)

Q: Change ownership of /var/www to user nginx and group nginx recursively

- `chown -R nginx:nginx /var/www`
 - `-R` applies changes to all files and subdirectories
-

5. User Management

Q: Create user santosh and add to sudo group

- `useradd santosh`
- `passwd santosh`
- `usermod -aG sudo santosh`
- On RHEL/CentOS, use wheel instead of sudo group

Q: Block a user temporarily without deleting account

- `passwd -l username`
 - `-l` locks the account; use `passwd -u` to unlock
-

6. Process Management

Q: Python app is consuming high CPU. Find and kill it

- `top` (then press P to sort by CPU) or `ps aux | grep python`
- `kill <PID>` or `kill -9 <PID>` for force kill
- `kill -9 (SIGKILL)` should be last resort; try `kill (SIGTERM)` first

Q: Check which process is using port 8080

- `ss -tulnp | grep 8080`
- `netstat -tulnp | grep 8080` (on older systems)
- `ss` is modern replacement for `netstat`

Q: Find which process is causing high disk I/O

- `iostat` (if available)
- Alternative: `iostat -x 2` or `pidstat -d 2`
- *May need to install iotop: `apt install iotop` or `yum install iotop`*

Q: Identify memory bottleneck

- `free -h`
 - `vmstat 2`
 - `top` → press M to sort by memory
 - `vmstat 2` updates every 2 seconds; watch 'si/so' for swap activity
-

7. Service Management (systemd)

Q: Check Nginx status, restart it, and view last 50 logs

- `systemctl status nginx`
- `systemctl restart nginx`
- `journalctl -u nginx -n 50`
- *Use `systemctl reload nginx` to apply config changes without downtime*

Q: Apply Nginx configuration changes without disconnecting active users

- `systemctl reload nginx`
 - *Reload gracefully reloads config; restart would briefly interrupt connections*
-

8. Scheduling

Q: Schedule a script to run daily at 6 PM

- `crontab -e`
 - Add: `0 18 * * * /opt/scripts/backup.sh`
 - *Format: minute hour day month weekday command*
-

9. Networking & Connectivity

Q: Check if server can reach Google DNS

- `ping 8.8.8.8`
- `curl -I https://google.com` (HTTP check)
- *ping tests ICMP; curl tests actual HTTP connectivity*

Q: Port 8080 is not reachable from outside, app is running. Debug steps

- `ss -tulpen | grep 8080` to see listener and bind IP
- `netstat -tulpen | grep 8080` (older systems)

- Check firewall: `iptables -L -n` or `firewall-cmd --list-all` or `ufw status`
- Test locally: `curl localhost:8080` and `curl <server-ip>:8080`
- *Check if app binds to 127.0.0.1 (localhost only) vs 0.0.0.0 (all interfaces)*

Q: Test a REST API endpoint (POST request) with JSON data

- `curl -X POST -H "Content-Type: application/json" -d '{"key":"value"}' http://api-url`
- *-H sets headers, -d sends data*

Q: Test if an API endpoint is working

- `curl -I https://api.example.com/status`
- *-I fetches headers only (HEAD request)*

Q: Check DNS record

- `dig google.com`
- `nslookup google.com`
- *dig provides more detailed output; nslookup is more portable*

Q: Check if port 3306 is open on remote host

- `nc -zv host 3306`
 - Alternative: `telnet host 3306`
 - *-z = zero I/O mode (scanning), -v = verbose*
-

10. Firewall Management

Q: Allow only SSH port 22 and block all other incoming traffic

- `ufw allow 22`
 - `ufw default deny incoming`
 - `ufw enable`
 - *On RHEL/CentOS, use `firewall-cmd` or `iptables` instead*
-

11. SSH & Authentication

Q: Generate SSH key pair & copy public key to remote server

- `ssh-keygen -t rsa -b 4096`
 - `ssh-copy-id user@server`
 - *Modern systems: use `ssh-keygen -t ed25519` for better security*
-

12. Package Management

Q: Update package index, upgrade installed packages, install htop, remove nginx (Ubuntu)

- `sudo apt update` (refreshes package list)
- `sudo apt upgrade` (upgrades packages)
- `sudo apt install htop`
- `sudo apt remove nginx` or `sudo apt purge nginx` (purge removes config files)

Q: Update system packages & upgrade security patches

- Ubuntu: `apt update && apt upgrade -y`
- RHEL/CentOS: `yum update -y` or `dnf update -y`

Q: Remove package nginx including dependencies

- Ubuntu: `apt purge nginx` or `apt autoremove nginx`
 - RHEL: `yum remove nginx -y`
-

13. Disk & Storage

Q: Check disk usage and find which folder consumes most space

- `df -h` (disk space by filesystem)
 - `du -sh * | sort -h` (directory usage, human-readable sorted)
 - Alternative: `du -h --max-depth=1 / | sort -h`
 - *Run du from the directory you want to analyze*
-

14. Logging & Log Analysis

Q: Check last 100 lines of system logs

- `journalctl -n 100`
- Alternative: `tail -100 /var/log/syslog` (older systems)

Q: Search for "error" in Nginx logs

- `grep -i Error /var/log/nginx/error.log`
- *-i makes search case-insensitive*

Q: Find total number of failed login events

Login_fail

Login fail

Login. FAIL

```
cat a.log | grep -I login | grep -I fail | wc -l
```

- `grep "Failed password" /var/log/auth.log | wc -l`
 - *May need to check /var/log/secure on RHEL/CentOS*
-

15. Redirection & Background Processes

Q: Explain `>`, `>>`, `2>`, and `2>&1`

- `>` : Overwrite stdout to file (e.g., `cmd > out.log`)
- `>>` : Append stdout to file (e.g., `cmd >> out.log`)
- `2>` : Send stderr to file (e.g., `cmd 2> err.log`)
- `2>&1` : Redirect stderr to same place as stdout (e.g., `cmd > all.log 2>&1`)
- *File descriptor: 1=stdout, 2=stderr, 0=stdin*

Q: Redirect errors to a file while keeping output on screen

- `command 2>error.log`
- To see both: `command 2>&1 | tee output.log`

Q: Start a long-running process on remote SSH that won't die if SSH drops

- `nohup ./long_task.sh > output.log 2>&1 &`
- Alternative: Use `screen` or `tmux` sessions
- `nohup` ignores *HUP* signal, & runs in background

Q: Run script in background and ensure it keeps running after logout

- `nohup script.sh &`
 - Alternative: `screen -dmS session script.sh`
-

16. Environment Variables

Q: Difference between `env`, `export`, and `.bashrc` vs `.profile`

- `env` : Print environment or run command with modified environment
- `export VAR=value` : Make variable available to child processes
- `.profile` : Login shell initialization (environment, PATH) - runs once at login
- `.bashrc` : Non-login interactive shell initialization (aliases, functions) - runs for each new shell
- *Login shells source .profile; interactive shells source .bashrc*

Q: Temporarily set an environment variable for current shell

- `export ENV=prod`
 - *Variable persists only in current shell session*
-

17. SSL/TLS

Q: Generate a private key + CSR

- `openssl genrsa -out server.key 2048`
 - `openssl req -new -key server.key -out server.csr`
 - *CSR = Certificate Signing Request, sent to CA for certificate issuance*
-